CM12002
Computer Systems Architectures

Instructions

Fabio Nemetz

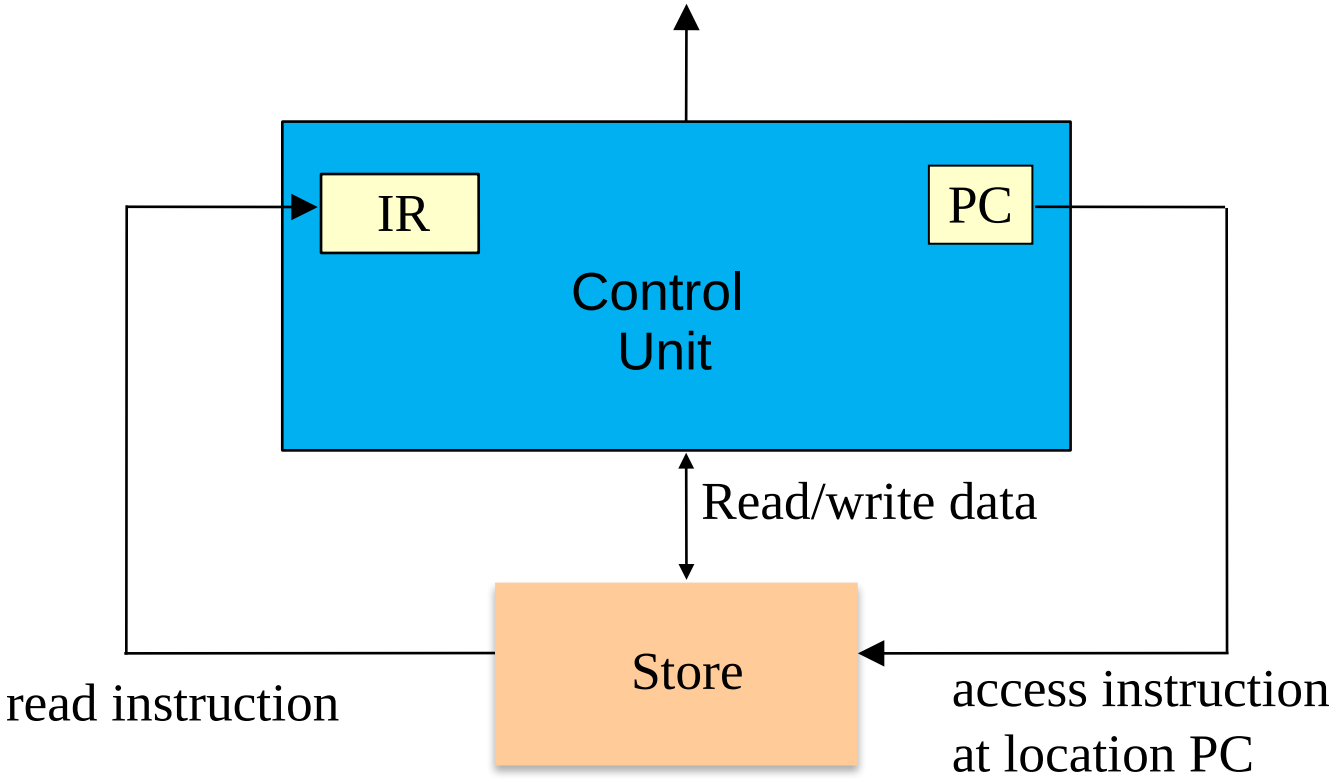
Instruction Execution

In this lecture:

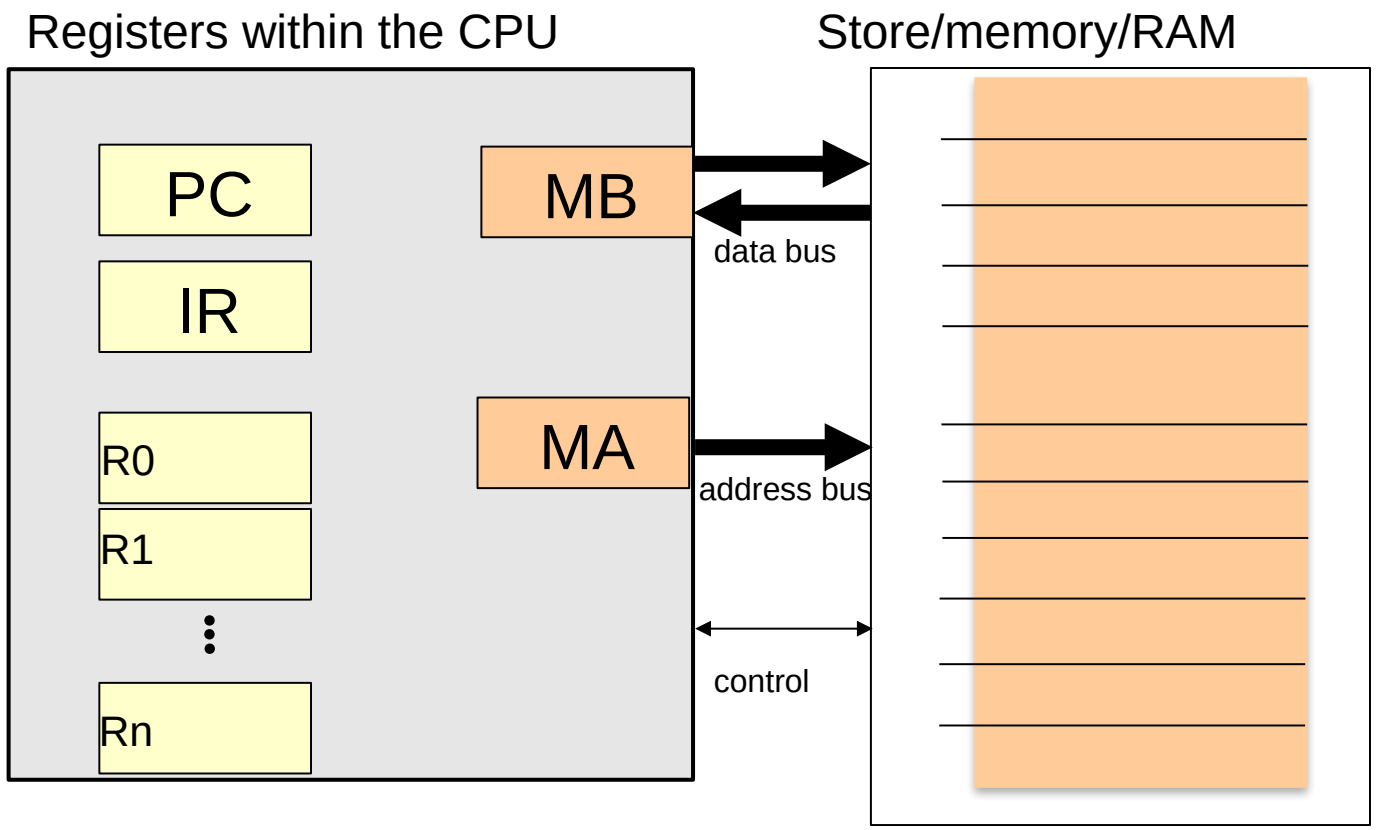
- The basic fetch-execute cycle (aka fetch-decode-execute), and how instructions are coded

- An example of a cycle.

Recap: The final model for instruction access



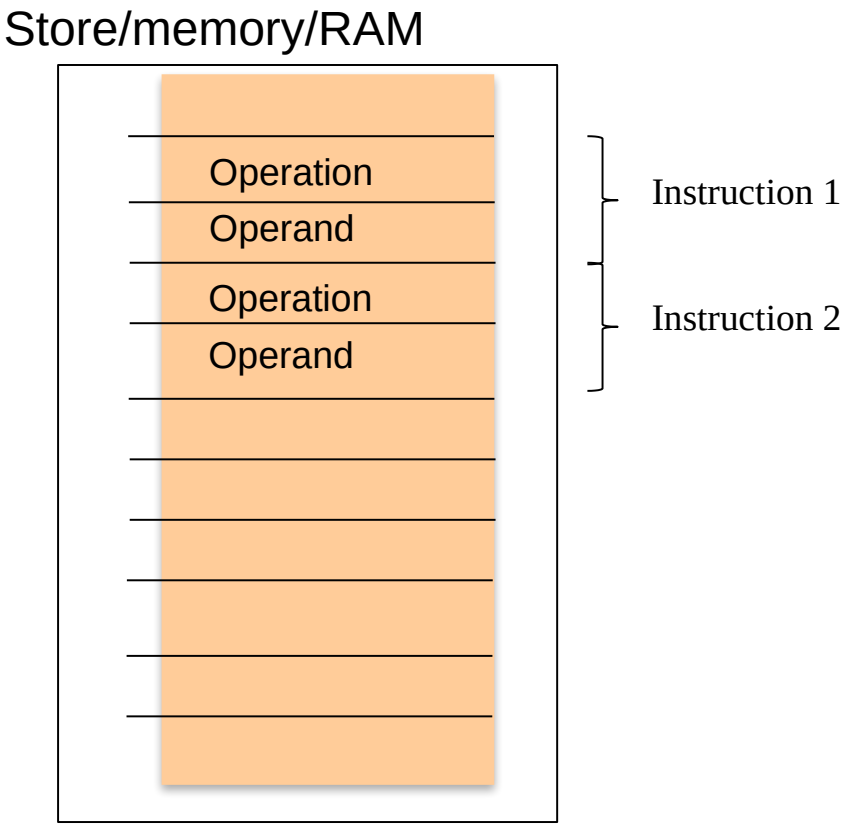
The final model with registers



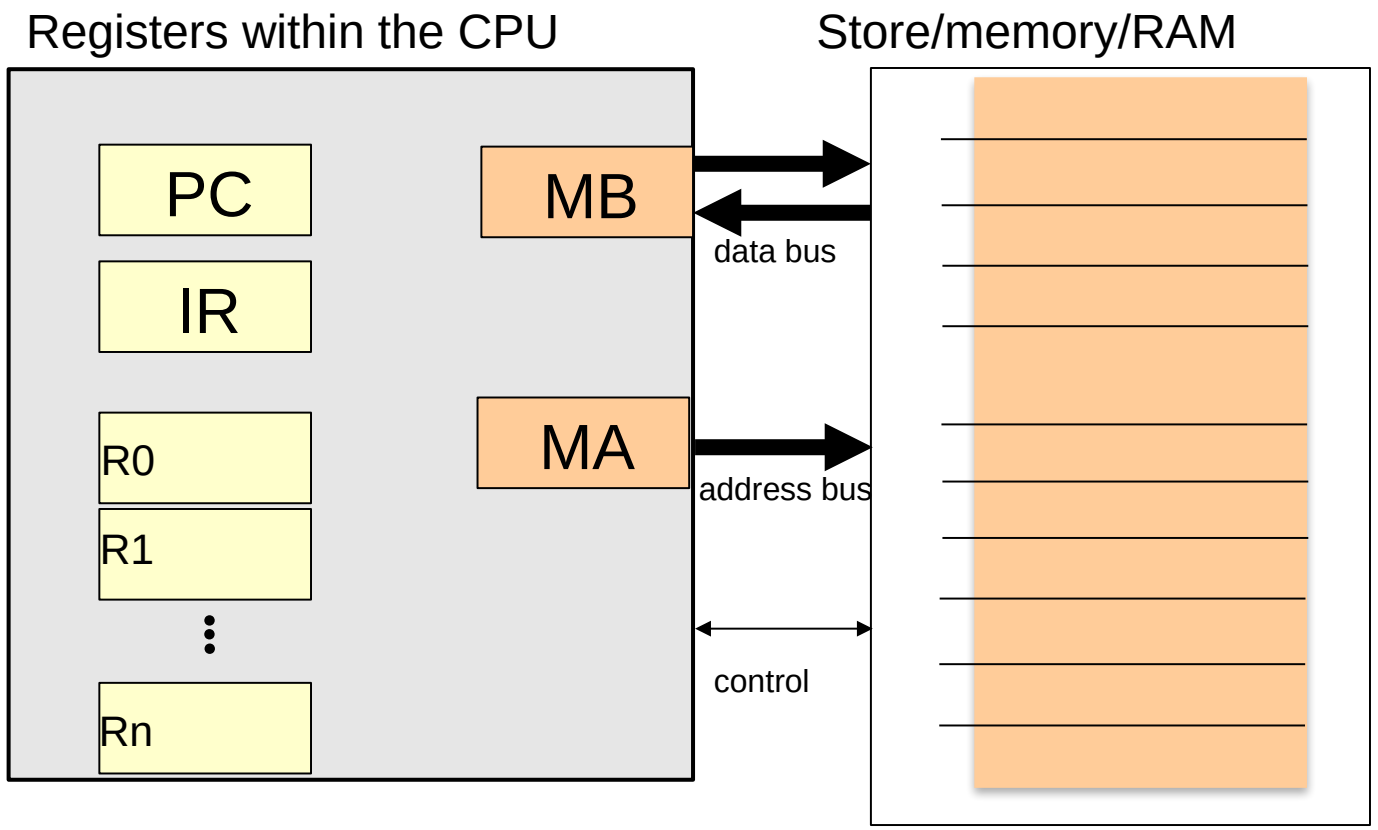
PC: Program counter
IR: Instruction Register
Rn: Data Register n
MB: Memory Buffer
MA: memory Address

Store

A program will be a sequence of Instructions: Operations and Operands



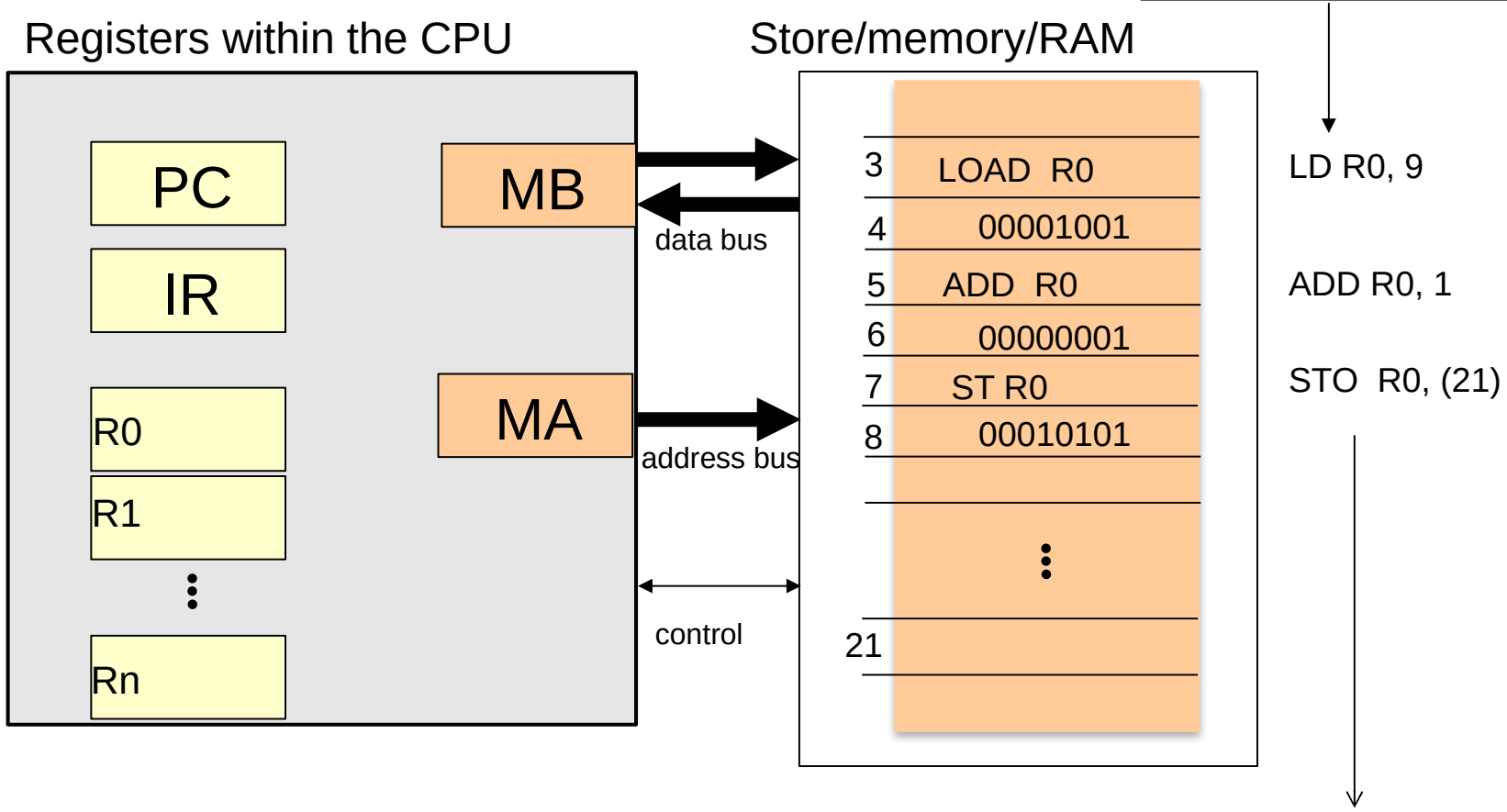
Execution of instructions



PC: Program Counter
IR: Instruction Register
Rn: Data Register n
MB: Memory Buffer
MA: memory Address

Execution of instructions

You could write #9 to say it's value 9 (and not address 9).
In certain architectures, the mnenomic changes, like to LDI R0, 9



1. Load 9 into Register 0
2. Increment R0 (add 1)
3. Store result into the cell whose address is 21

PC: Program Counter
IR: Instruction Register
Rn: Data Register n
MB: Memory Buffer
MA: memory Address

Execution of instructions

To carry out an instruction:

- it must first be *fetched* from store to the control unit.
- then *interpreted* (or *decoded*) to determine what operation is to be performed and how the operands are to be accessed.
- the operands must be fetched from store to the ALU
- the operation upon them must be executed, using control signals supplied by the control unit.

Execution of instructions

The instruction cycle

1. Fetch phase: bringing the instruction from store to the instruction register.

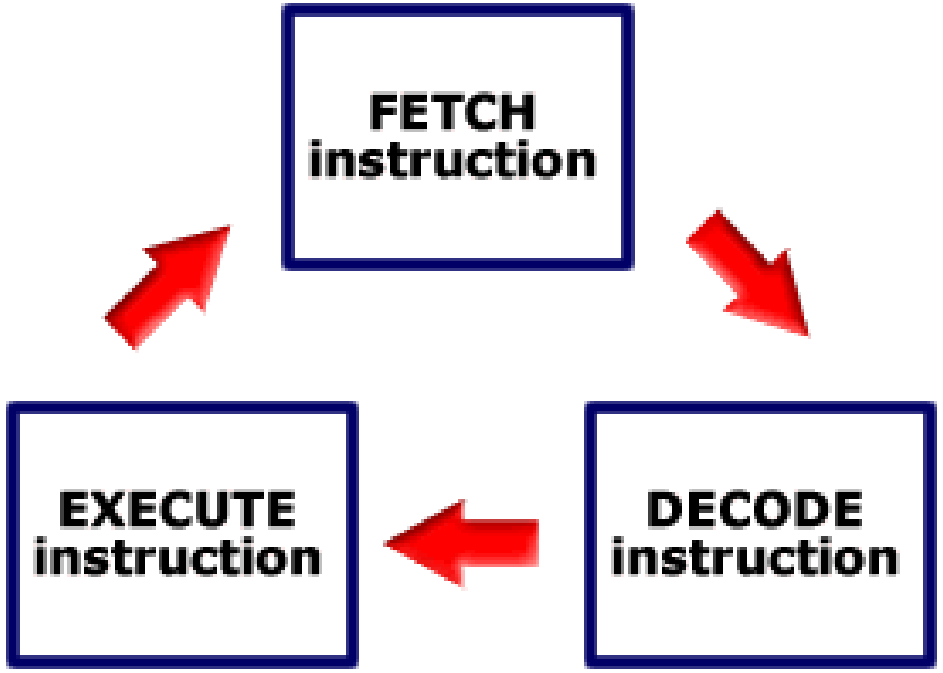
This is the same for all instructions.

2. *Execution phase: decoding* the instruction, accessing the operands (if any) and carrying out the operation.

This phase varies from instruction to instruction, depending upon the operation.

The machine cycle (fetch execute cycle)

Fetch (and Increment PC) → Decode → Execute



In this diagram we separate Decode and Execute phases

Execution of instructions: FETCH

We can now see how this might be carried out:

Note: at the start of a fetch phase, PC points to the next instruction to be executed

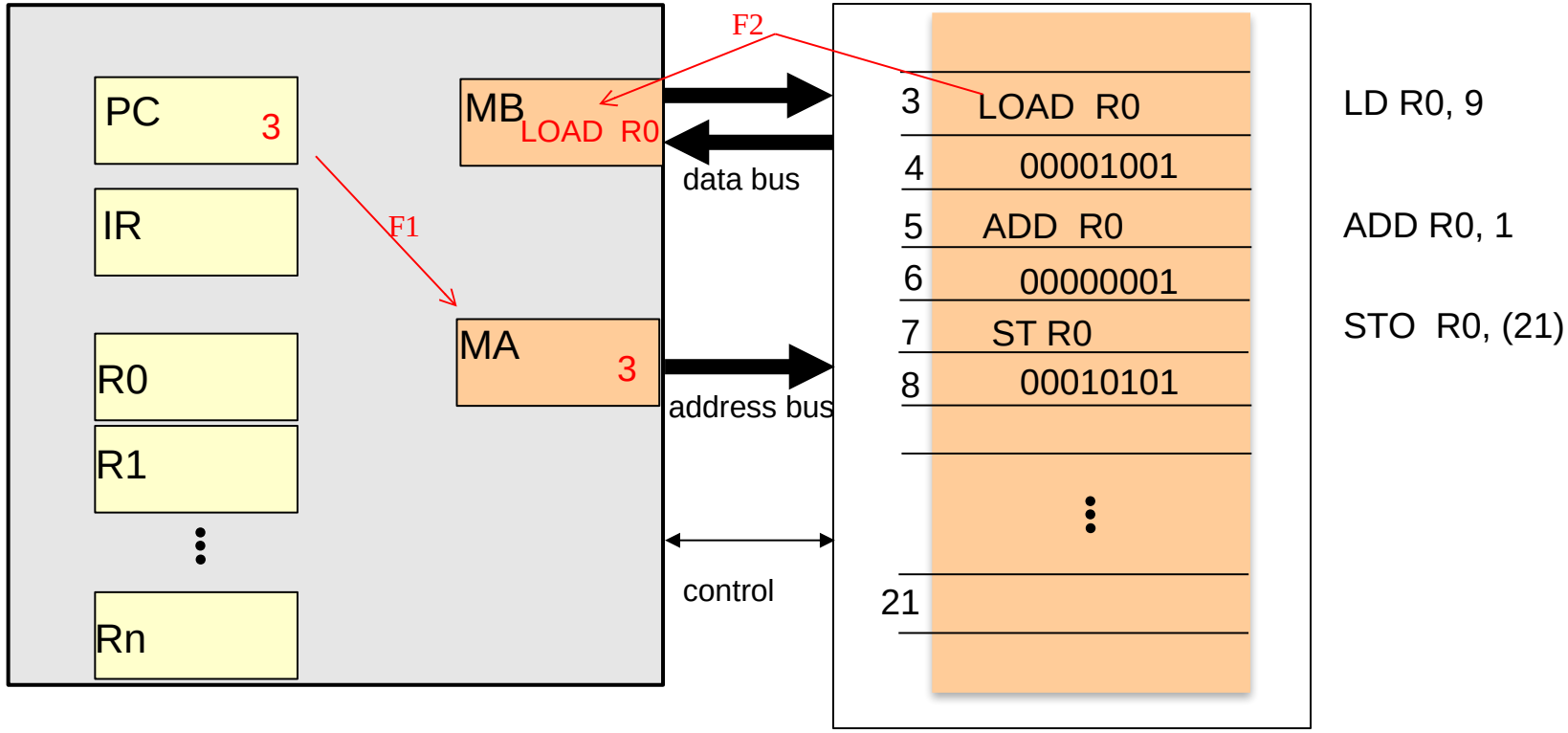
F1. Contents of the PC are placed in the MA:
this sets the store location of the next instruction;

F2. Contents of the cell whose address is in MA are placed in the MB:
this gets the contents of the cell;

F3. Contents of the MB are placed in the IR: the instruction is now ready for decoding; and

F4. Increment PC:
so that the next instruction in sequence will be accessed next.

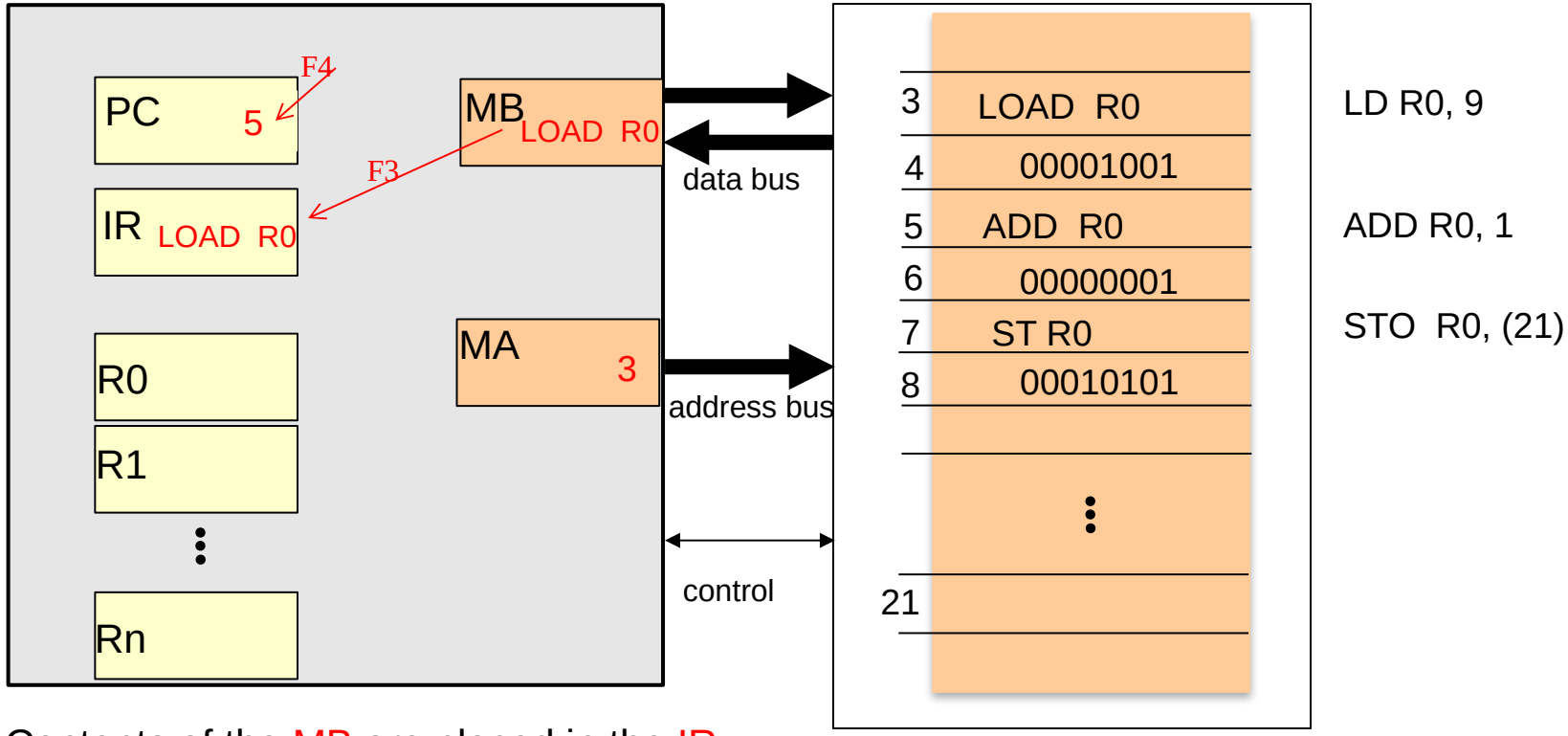
Execution of instructions: FETCH



F1. Contents of the PC are placed in the MA:
this sets the store location of the next instruction;

F2. Contents of the cell whose address is in MA are placed in the MB:
this gets the contents of the cell;

Execution of instructions: FETCH



F3. Contents of the MB are placed in the IR:
the instruction is now ready for decoding;

F4. Increment PC:
so that the next instruction in sequence will be accessed next.
(in this case, instruction takes 2 bytes)

Execution of instructions: EXECUTION

Execution phase

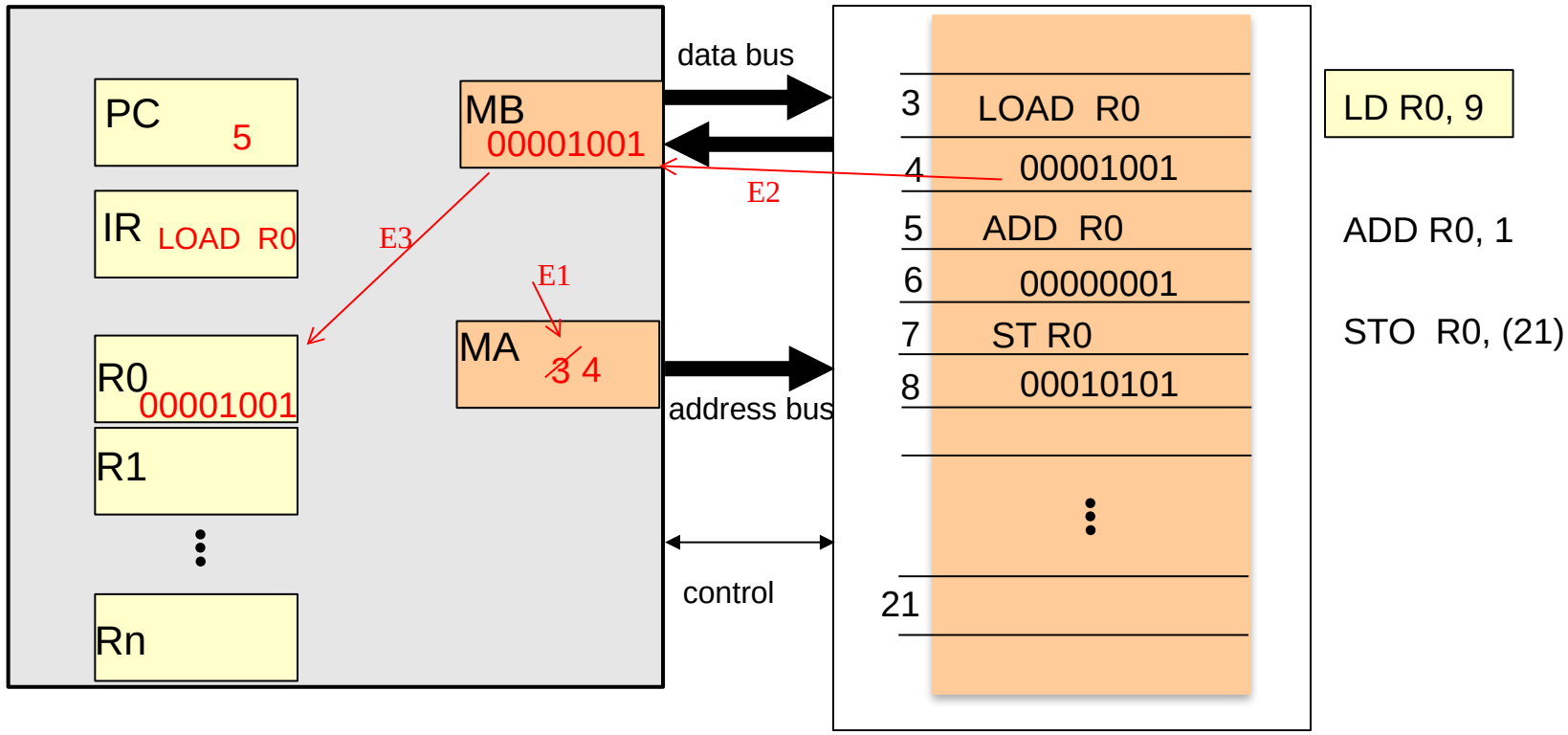
This depends upon the operation specified by the decoded contents of the IR. In our example, current instruction specifies *copying the contents of the next cell (relative to the operation) to data register R0*. We could do this as follows:

E1. Increment MA;

E2. Contents of the cell whose address is in MA are placed in the MB;

E3. Contents of MB are placed in the data register

Execution of instructions: EXECUTION



E1. Increment MA;
E2. Contents of the cell whose address is in MA are placed in the MB;
E3. Contents of MB are placed in the data register

Execution of instructions

Execution phase (example 2)

At home, try to understand this example:

Now let us assume that, on the same machine, the current instruction specifies *adding the contents of a cell at a specific location to the data register R0*. We could do this as follows:

E1. Increment MA;

E2. Contents of the cell whose address is in MA are placed in the MB;

E3. Contents of MB are placed in the MA

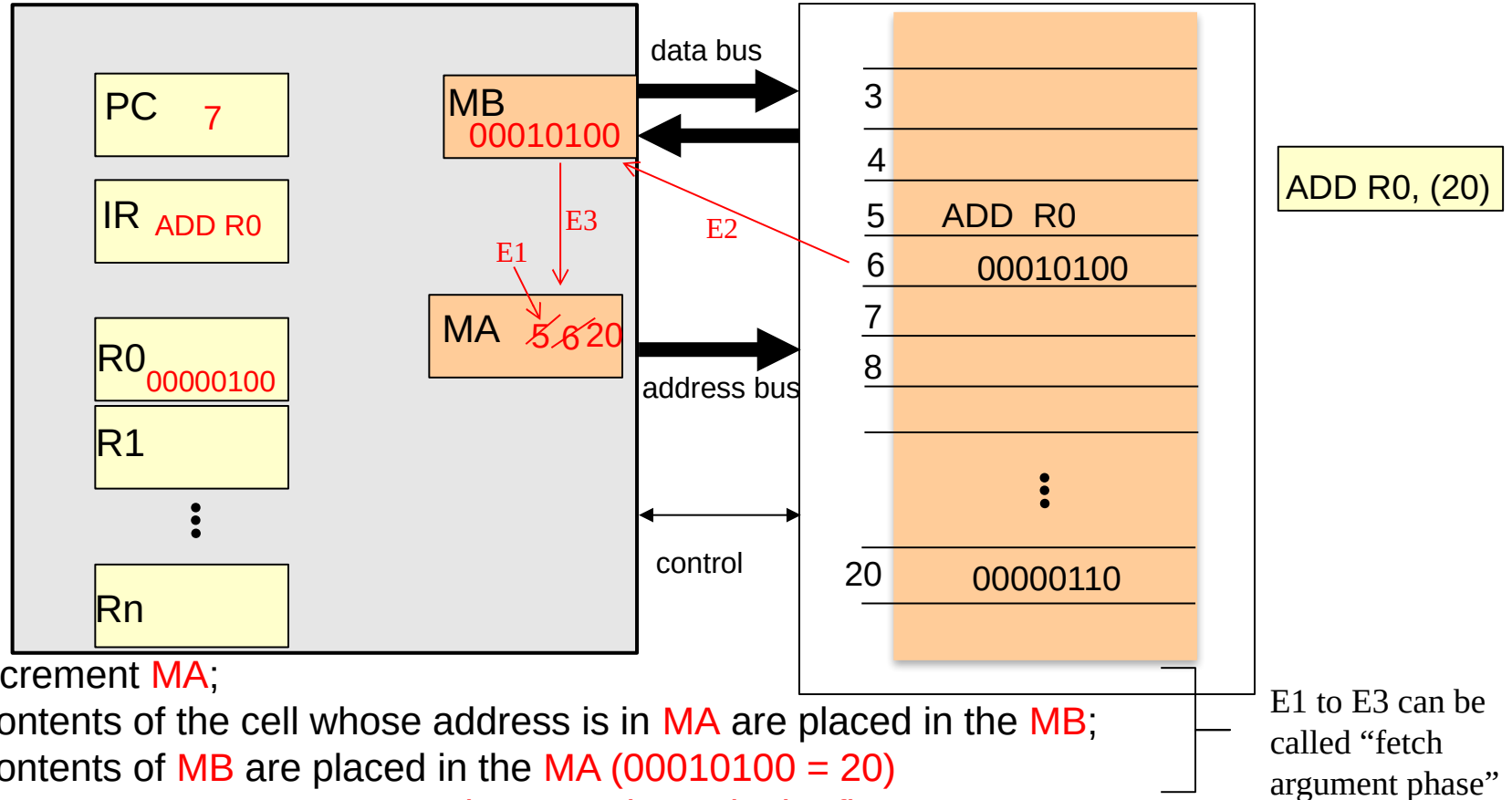
E4. Contents of the cell whose address is in MA are placed in the MB;

E5. Contents of MB are added to the existing contents of register R0;

ADD R0, (20) --- add the contents of location (address) 20 to register R0

Execution of instructions: EXECUTION

(we start assuming the fetch phase already happened)



E1. Increment MA;
E2. Contents of the cell whose address is in MA are placed in the MB;
E3. Contents of MB are placed in the MA (00010100 = 20)

E1 to E3 can be called "fetch argument phase"

----------E4 and E5 not shown in the figure ----------

E4. Contents of the cell whose address is in MA are placed in the MB; (not shown in the figure)
E5. Contents of MB are added to the existing contents of register R0; (not shown in the figure)

A real instruction set: MC68000

Example: the Motorola MC68000 architecture

We saw earlier that no real computer could manage with a one-byte instruction word, but used several bytes.

This does not fundamentally alter the nature of instruction execution as we have described it.

Let us take the example of an addition operation that adds the 16-bit contents of a specified location to one of the eight data registers of this machine (we shall choose register 5 for this example).

ADD $1202, D5 *(equivalent to the previous example ADD D5, ($1202))*

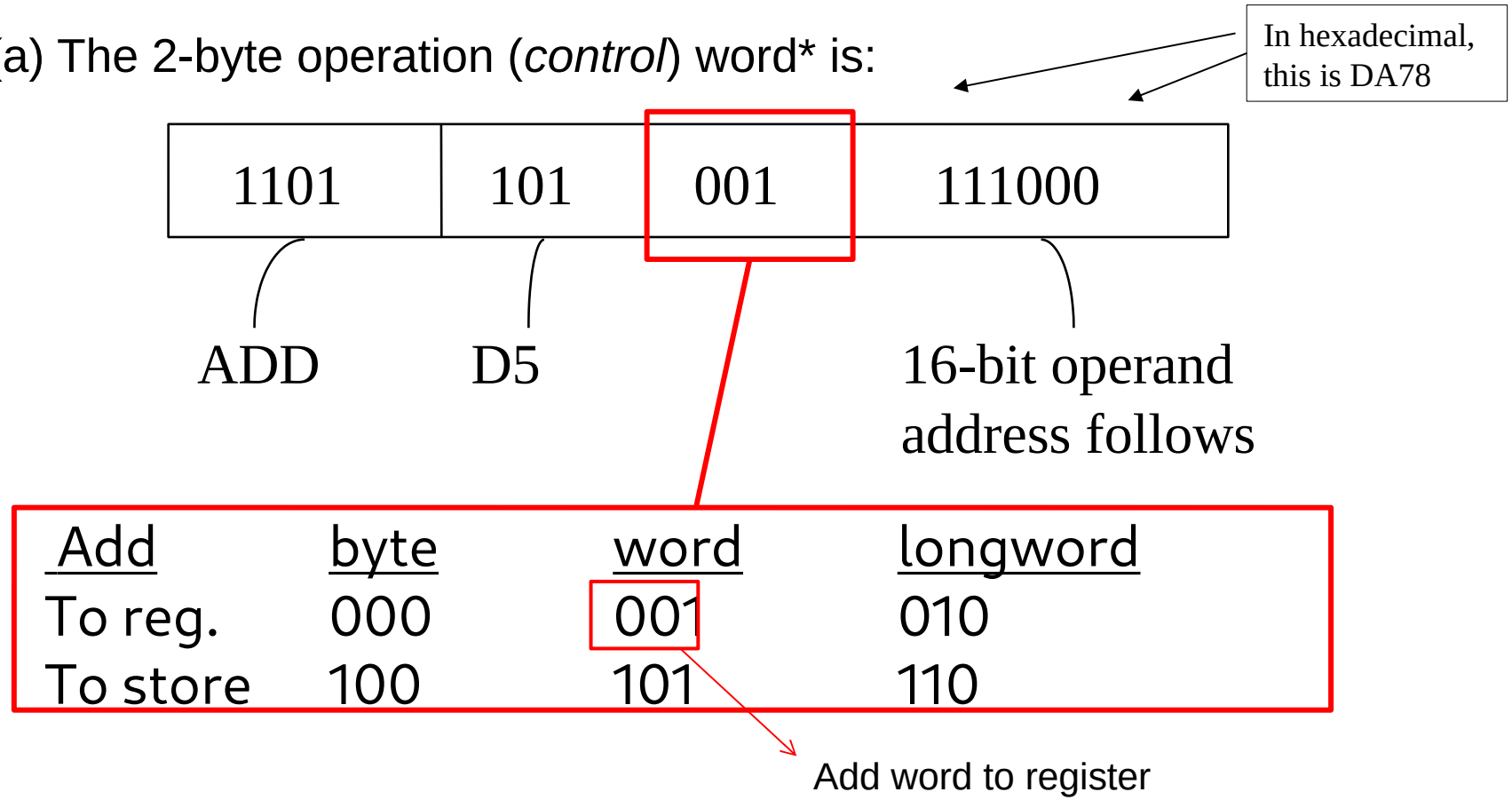
This instruction will be stored in 4 bytes.

-the '$' symbol indicates the number is in hexadecimal – it's a convention for Motorola processors – could also use prefix '0x' or suffix 'h' (e.g., 0x1202 or 1202h)

A real instruction set: MC68000

ADD $1202, D5

(a) The 2-byte operation (*control*) word* is:



*For the MC68000, a word means 16 bits – a word means "the natural chunk of data for this processor

A real instruction set: MC68000

ADD $1202, D5

(b) The 2-byte operand address (*extension*) word is:

0001001000000010	= $1202

This is simply the store address in binary.

The four bytes (let us assume they are stored at the address $1000 hexadecimal) are thus (in hexadecimal):

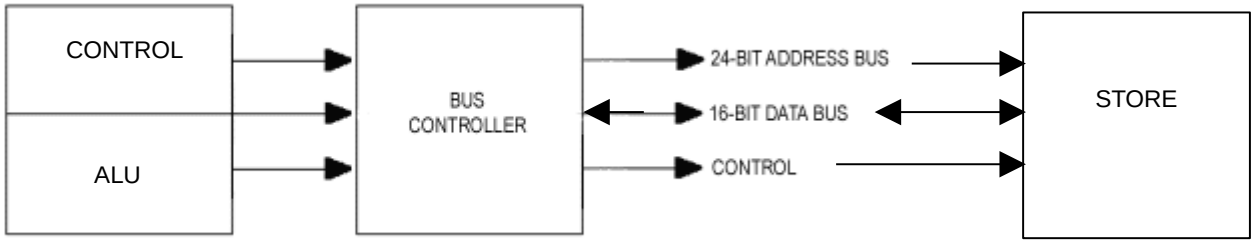
Address $1000:	$DA78
Address $1002:	$1202

Easier to read hex than binary

A real instruction set: MC68000

The 68000 bus architecture

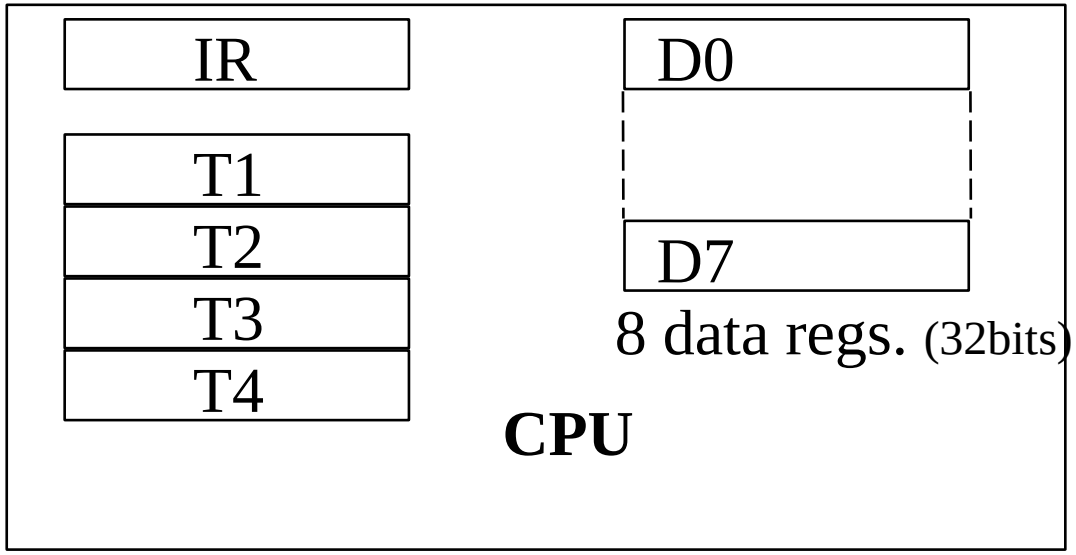
Store and I/O devices are connected to the CPU via a single electronic “pathway”, or *bus*, that contains lines for addresses, data and control signals:



A real instruction set: MC68000

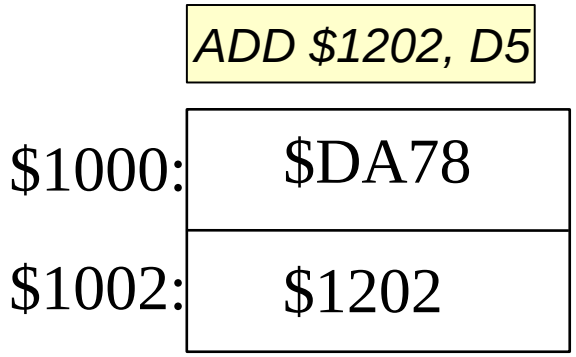
In the 68000 architecture, the CPU contains

- a 16-bit (2-byte) IR (for the instruction *control* word) and
- four other inaccessible temporary registers which we can call T1, T2, T3 and T4 (for the instruction *extension* words, of which there are a *maximum* of four):



A real instruction set: MC68000

Execution of the addition instruction (a)



Note that (X) is used to denote the *contents* of X and we take it that (PC) is $1000 (1000 hexadecimal) to start.

1. Fetch phase:

- (PC) → placed on the address lines of the bus [$1000];
- (PC) + 2 → PC [$1002];
- Read signal is placed on the control lines of bus;
- Begin read from store, placing data on the data lines of bus [$DA78];
- Data lines on bus → IR [$DA78];
- (IR) -> decode logic in CPU.

A real instruction set: MC68000

Execution of the addition instruction (b)

2. Execute phase:

- (PC) → placed on the address lines of bus [$1002];
- (PC) + 2 → PC [$1004];
- Read signal is placed on the control lines of bus;
- Begin read from store, placing data on the data lines of bus [$1202];
- Data lines on bus → T1 [$1202];

At this point, the operand address having been accessed from store, the operation itself can be carried out as follows on the next slide...

A real instruction set: MC68000

Execution of the addition instruction (c)

3. Execute phase continues:

- (T1) -> placed on the address lines of bus [$1202]
- Read signal is placed on the control lines of bus;
- Begin read from store, placing data on the data lines of bus [contents of location $1202];
- Add the contents of the data lines of bus to the lowest order 16 bits of the 32-bit data register 5 in the ALU.

At the end of this operation the (PC) points to the next instruction, whose control word should be found in location $1004.

Homework

1. Watch these 4-video playlist: http://goo.gl/U2bLIL
   - Bear in mind it is a different architecture – but very useful to consolidate your learning (we'll discuss Immediate addressing and absolute addressing in the next lecture) – Total time 25mins (first 2, 15mins)

2. Check Execution Phase – example 2 (slides 17 and 18)

3. Go over the fetch and execution phases of *ADD $1202, D5 for the MC68000 (slides 24 to 26)*

Summary

In this lecture, we have:

- Considered the *fetch-execute* cycle, and the makeup of instructions in greater detail,
- Worked an example of a cycle on the Motorola 68000 architecture.

Next lecture:

- More about coding instructions: different modes for representing addresses in the store.